

# Lecture 10: Multi-layer Perceptron

## Introduction to Machine Learning

Sajjad Amini

Department of Electrical Engineering  
Sharif University of Technology

# Contents

- 1 Approach Definition
- 2 Perceptron Algorithm
- 3 Multi-layer Perceptron
- 4 Differentiable MLPs
- 5 Activation Functions
- 6 Backpropagation

Except explicitly cited, the reference for the material in slides is:

- Murphy, K. P. (2022). *Probabilistic machine learning: an introduction*. MIT press.

# Section 1

## Approach Definition

# Approach Definition

## Linear Models

- Multinomial logistic regression assume the following model:

$$p(y|\mathbf{x}, \mathbf{w}) = \text{Cat}(y|\mathcal{S}(\mathbf{W}\mathbf{x}))$$

- Linear regression assume the following model:

$$p(y|\mathbf{x}, \mathbf{w}, \sigma^2) = \mathcal{N}(y|\mathbf{w}^T \mathbf{x}, \sigma^2)$$

One shared feature among both model is linearity.

## Increasing Flexibility

To increase flexibility, we can replace input features  $\mathbf{x}$  with transformed version  $\phi(\mathbf{x})$  known as **basis function expansion**. Then we have the following model:

$$f(\mathbf{x}; \mathbf{W}) = \mathbf{W} \phi(\mathbf{x})$$

The above model is linear in weight matrix  $\mathbf{W}$  which makes the estimation easy.

## Toward Automating Transformation (Deep Learning)

- Parameterizing Transformation:  $\phi(\mathbf{x}) \Rightarrow \phi(\mathbf{x}, \boldsymbol{\theta})$ 
  - $\phi([x_1, x_2]^T; [\theta_1, \theta_2]^T) = [(\theta_1 + x_1)^2 + (\theta_2 + x_2)^2, \sin(\theta_1 x_1 + \theta_2 x_2)]$
- Applying the transformations in a hierarchical manner:

$$\mathbf{z}_1 = \phi_1(\mathbf{z}_0, \boldsymbol{\theta}_1), \mathbf{z}_0 = \mathbf{x}$$

$$\mathbf{z}_2 = \phi_2(\mathbf{z}_1, \boldsymbol{\theta}_2)$$

$$\vdots$$

$$\mathbf{z}_L = \phi_L(\mathbf{z}_{L-1}, \boldsymbol{\theta}_L)$$

Altogether we have  $\mathbf{z}_L = \phi(\mathbf{x}, \boldsymbol{\theta}) = \phi_L(\phi_{L-1}(\dots \mathbf{z}_0 \dots, \boldsymbol{\theta}_{L-1}), \boldsymbol{\theta}_L)$  where:

$$\boldsymbol{\theta} = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \dots, \boldsymbol{\theta}_L)$$

and  $\phi_l(\cdot, \boldsymbol{\theta}_l)$  is transformation at layer  $l$ .

## Section 2

# Perceptron Algorithm

# Perceptron Algorithm

## Binary Logistic Regression

In binary logistic regression, the posterior distribution over labels is modeled as:

$$p(y|\mathbf{x}, \mathbf{w}) = \text{Ber}(y|\sigma(\mathbf{w}^T \mathbf{x}))$$

## Perceptron

Perceptron is deterministic version of logistic regression (Why??) where the posterior is modeled as:

$$p(y|\mathbf{x}, \mathbf{w}) = \text{Ber}(y|H(\mathbf{w}^T \mathbf{x}))$$

where  $H(\mathbf{w}^T \mathbf{x}) = \mathbb{I}(\mathbf{w}^T \mathbf{x} \geq 0)$  is heaviside step function.

# Perceptron Algorithm

## Learning Algorithm

The update rule proposed by Rosenblatt for Perceptron is:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t(\hat{y}_n - y_n)\mathbf{x}_n$$

We have seen before the update rule for BLR as:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t(\mu_n - y_n)\mathbf{x}_n$$

## Perceptron Vs BLR

- Perceptron:
  - No need to compute the probability
  - Convergent when the problem is linearly separable
- BLR
  - $\mu$  is needed for update
  - Always convergent to minimizer of MLE

# Perceptron Algorithm

## Intuition

Consider Perceptron learning algorithm as:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t(\hat{y}_n - y_n)\mathbf{x}_n$$

Four different cases can occur (assume  $\eta_t = 1$ ):

$$y_n = 1, \hat{y}_n = 0 \Rightarrow \mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{x}_n$$

$$y_n = 0, \hat{y}_n = 1 \Rightarrow \mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{x}_n$$

$$y_n = 0, \hat{y}_n = 0 \Rightarrow \mathbf{w}_{t+1} = \mathbf{w}_t$$

$$y_n = 1, \hat{y}_n = 1 \Rightarrow \mathbf{w}_{t+1} = \mathbf{w}_t$$

## Section 3

# Multi-layer Perceptron

# Perceptron Learning Limitation

## XOR Function

Assume XOR function defined as:

$$y = x_1 \oplus x_2 = \begin{cases} 0 & \text{if } x_1 = 0, x_2 = 0 \\ 0 & \text{if } x_1 = 1, x_2 = 1 \\ 1 & \text{if } x_1 = 1, x_2 = 0 \\ 1 & \text{if } x_1 = 0, x_2 = 1 \end{cases}$$

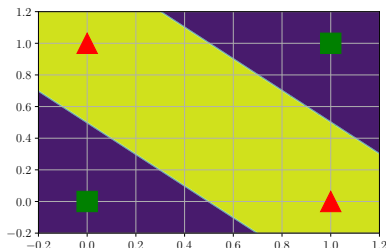


Figure: XOR problem

# Quest for Linearly Separable Features

## XOR Function

Assume the following transformations:

$$h_1 = x_1 \wedge x_2 = \mathbf{w}_1^T \mathbf{x} + b_1, \begin{cases} \mathbf{w}_1 = [1, 1]^T \\ b_1 = -1.5 \end{cases}$$

$$h_2 = x_1 \vee x_2 = \mathbf{w}_2^T \mathbf{x} + b_2, \begin{cases} \mathbf{w}_2 = [1, 1]^T \\ b_2 = -0.5 \end{cases}$$

Then we can show that

$$y = \bar{h}_1 \wedge h_2 = \overline{(x_1 \wedge x_2)} \wedge (x_1 \vee x_2) = \mathbf{w}_3^T \mathbf{x} + b_3, \begin{cases} \mathbf{w}_3 = [-1, 1]^T \\ b_3 = -0.5 \end{cases}$$

The resulting model is called *Multi-Layer Perceptron* (MLP).

## XOR Function

The final model consist of three Perceptrons, denoted  $h_1$ ,  $h_2$  and  $y$ .

- Hidden unit:  $h_1$  and  $h_2$  are hidden units (Perceptrons) since they are not observed in the training data.
- Output unit:  $y$  is output unit (Perceptron).

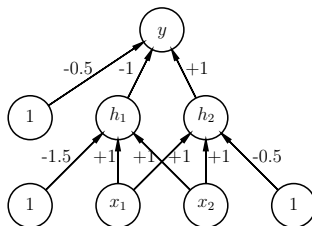


Figure: MLP model for XOR problem

## Section 4

# Differentiable MLPs

## Problem with MLPs

Training MLP as a stack of Perceptrons is difficult due to non-differentiable Heaviside function.

## Differentiable MLPs

Differentiable MLPs are classical MLPs while Heaviside function is replaced with a differentiable function  $\varphi : \mathbb{R} \rightarrow \mathbb{R}$  known as *Activation Function*.

## Model

Assume the following definitions:

|                    |                                  |
|--------------------|----------------------------------|
| $l$                | Layer number                     |
| $z_l$              | Hidden units at layer $l$        |
| $\varphi_l(\cdot)$ | Activation function at layer $l$ |
| $K_l$              | Feature dimension at layer $l$   |

Then the mapping in layer  $l$  is:

$$z_l = \phi_l(z_{l-1}, \theta_l) = \varphi_l(\mathbf{b}_l + \mathbf{W}_l z_{l-1})$$

Note that the quantity passed to activation function is called *pre-activations* defined as:

$$\mathbf{a}_l = \mathbf{b}_l + \mathbf{W}_l z_{l-1}$$

## MLP

The term MLP refer to the differentiable MLP rather than non-differentiable version based on Heaviside step function.

## Section 5

# Activation Functions

# Activation Functions

## Linear Activation Functions

Assume we select  $\varphi_l(a) = c_l a$ . Then the whole MLP becomes:

$$\mathbf{z}_1 = \varphi_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) = c_1 \mathbf{W}_1 \mathbf{x} + c_1 \mathbf{b}_1$$

$$\mathbf{z}_2 = \varphi_2(\mathbf{W}_2 \mathbf{z}_1 + \mathbf{b}_2) = c_2 \mathbf{W}_2 \mathbf{z}_1 + c_2 \mathbf{b}_2 = \underbrace{c_1 c_2 \mathbf{W}_2 \mathbf{W}_1}_{\mathbf{W}_{12}} \mathbf{x} + \underbrace{c_1 c_2 \mathbf{W}_2 \mathbf{b}_1 + c_2 \mathbf{b}_2}_{\mathbf{b}_{12}}$$

...

$$\mathbf{z}_L = \varphi_L(\mathbf{W}_L \mathbf{z}_{L-1} + \mathbf{b}_L) = \mathbf{W}_{1...L} \mathbf{x} + \mathbf{b}_{1...L}$$

Thus linear activation function reduces to regular linear model. Thus it is important to use nonlinear activation functions.

## Sample Activation Functions

- Sigmoid:

$$\varphi(a) = \sigma(a) = \frac{1}{1 + e^{-a}}$$

- Hyperbolic tangent:

$$\varphi(a) = \tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$$

- Rectified linear unit:

$$\varphi(a) = \text{ReLU}(a) = \max(a, 0) = aH(a)$$

# Sample Activation Functions

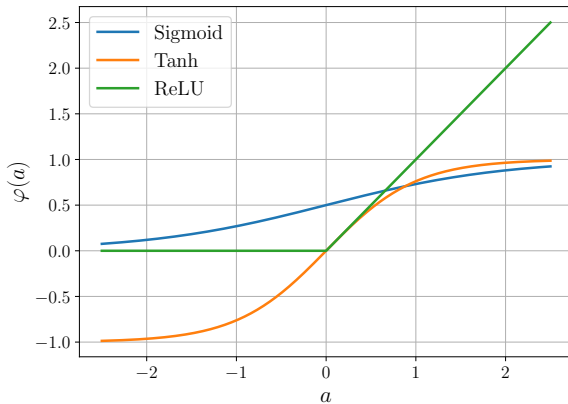


Figure: Sample Activation Functions

## Binary Classification

Consider a binary classification problem with  $y \in \{0, 1\}$  and  $\mathbf{x} \in \mathbb{R}^2$ . Assume MLP model with the following features:

- Two hidden layers as:

$$\mathbf{z}_1 = \tanh(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \begin{cases} \mathbf{x} \in \mathbb{R}^2 \\ \mathbf{W}_1 \in \mathbb{R}^{4 \times 2} \\ \mathbf{b}_1, \mathbf{z}_1 \in \mathbb{R}^4 \end{cases}$$
$$\mathbf{z}_2 = \tanh(\mathbf{W}_2 \mathbf{z}_1 + \mathbf{b}_2), \begin{cases} \mathbf{W}_2 \in \mathbb{R}^{3 \times 4} \\ \mathbf{b}_2, \mathbf{z}_2 \in \mathbb{R}^3 \end{cases}$$

- Output layer as:

$$a_3 = \mathbf{w}_3^T \mathbf{z}_2 + b_3, \begin{cases} \mathbf{w}_3 \in \mathbb{R}^3 \\ b_3, a_3 \in \mathbb{R} \end{cases}$$

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \text{Ber}(y|\sigma(a_3))$$

# Sample MLP

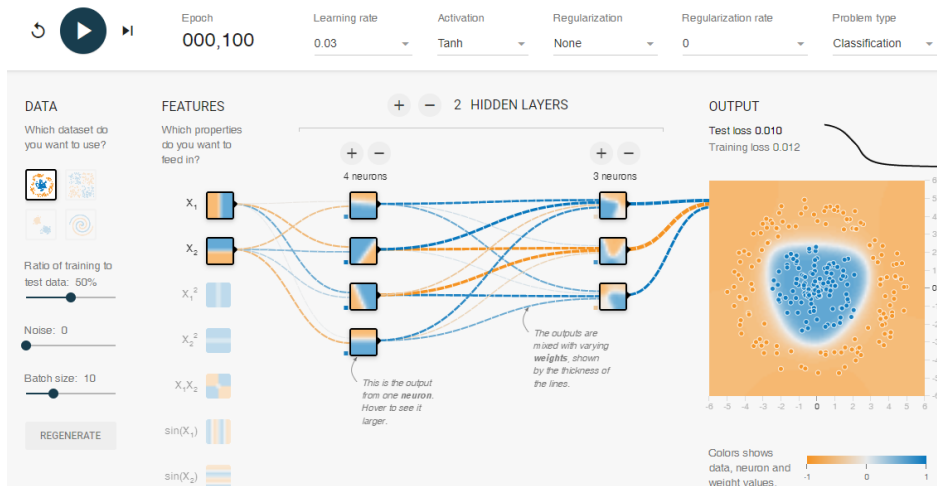


Figure: MLP Visualization

## Multi-class Classification

Consider classifying MNIST dataset [1] where  $y \in \{0, 1, \dots, 9\}$  and  $\mathbf{X} \in \mathbb{R}^{28 \times 28}$  ( we use the vectorized version of images as  $\mathbf{x} = \text{vec}(\mathbf{X}) \in \mathbb{R}^{784}$ ). Assume MLP model with the following features:

- Two hidden layers as:

$$\mathbf{z}_1 = \tanh(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \begin{cases} \mathbf{x} \in \mathbb{R}^{784} \\ \mathbf{W}_1 \in \mathbb{R}^{128 \times 784} \\ \mathbf{b}_1, \mathbf{z}_1 \in \mathbb{R}^{128} \end{cases}$$
$$\mathbf{z}_2 = \tanh(\mathbf{W}_2 \mathbf{z}_1 + \mathbf{b}_2), \begin{cases} \mathbf{W}_2 \in \mathbb{R}^{128 \times 128} \\ \mathbf{b}_2, \mathbf{z}_2 \in \mathbb{R}^{128} \end{cases}$$

- Output layer as:

$$\mathbf{a}_3 = \mathbf{W}_3 \mathbf{z}_2 + \mathbf{b}_3, \begin{cases} \mathbf{W}_3 \in \mathbb{R}^{10 \times 128} \\ \mathbf{b}_3, \mathbf{a}_3 \in \mathbb{R}^{10} \end{cases}$$
$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \text{Cat}(y|\mathcal{S}(\mathbf{a}_3))$$

# Sample MLP

Model: "sequential"

| Layer (type)      | Output Shape | Param # |
|-------------------|--------------|---------|
| flatten (Flatten) | (None, 784)  | 0       |
| dense (Dense)     | (None, 128)  | 100480  |
| dense_1 (Dense)   | (None, 128)  | 16512   |
| dense_2 (Dense)   | (None, 10)   | 1290    |

Total params: 118,282

Trainable params: 118,282

Non-trainable params: 0

Figure: MLP structure for MNIST classification

# Sample MLP

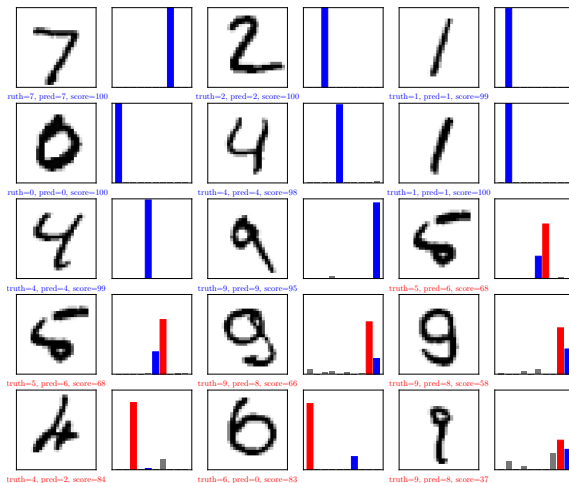


Figure: MLP results for MNIST classification after 1 epoch

# Sample MLP

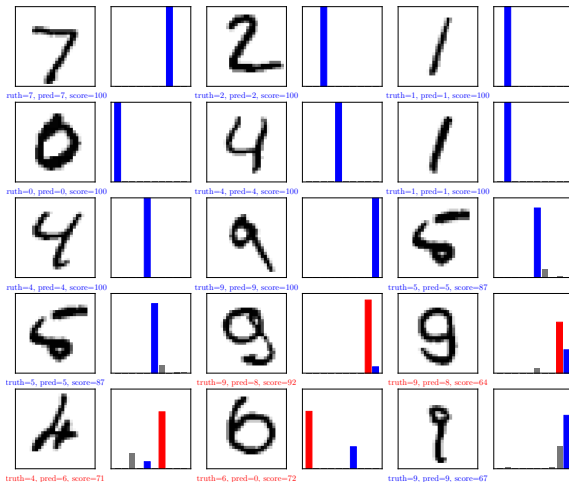


Figure: MLP results for MNIST classification after 2 epoch

## Section 6

# Backpropagation

# How train MLPs

## NLL for Multi-class Classification

For classification problem using MLP, we assume the following model:

$$p(y|\mathbf{x}; \boldsymbol{\theta}) = \text{Cat}(y | \underbrace{\mathcal{S}(\overbrace{\mathbf{W}_L^T \mathbf{z}_{L-1} + \mathbf{b}_L})^{\mathbf{a}_L}}_{\boldsymbol{\mu}_n})$$

Thus the NLL can be formulated as:

$$\begin{aligned} \text{NLL}(\boldsymbol{\theta}) &= -\log p(\mathcal{D}|\boldsymbol{\theta}) = -\log \prod_{n=1}^N \prod_{c=1}^C \mu_{nc}^{y_{nc}} = -\sum_{n=1}^N \sum_{c=1}^C y_{nc} \log \mu_{nc} \\ &= \sum_{n=1}^N \mathbb{H}(\mathbf{y}_n, \boldsymbol{\mu}_n) \end{aligned}$$

where  $\mathbf{y}_n$  is one-hot encoding of the label.

# How train MLPs

## NLL for Regression

For regression problem using MLP, we assume the following model:

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y | \overbrace{\mathbf{w}_L^T \mathbf{z}_{L-1} + b_L}^{a_L = \hat{y}}, \sigma^2)$$

Thus the NLL can be formulated as:

$$\begin{aligned} \text{NLL}(\boldsymbol{\theta}) &= -\log p(\mathcal{D}|\boldsymbol{\theta}) = -\log \prod_{i=1}^N p(y_n | \mathbf{x}_n, \boldsymbol{\theta}) \\ &= -\log \prod_{n=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y_n - \hat{y})^2\right) \\ &= \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - \hat{y}_n)^2 + \frac{N}{2} \log(2\pi\sigma^2) \end{aligned}$$

# Challenge

## Challenge

To minimize  $\text{NLL}(\boldsymbol{\theta})$ , you need to evaluate the gradient with respect to all parameters. Calculating the gradient when the MLP mapping is complex becomes challenging.

## MLP Structure

The structure of MLP is hierarchical. Thus we can reformulate  $\text{NLL}(\boldsymbol{\theta})$  in a hierarchical form. Assume a multi-class classification MLP with 2 hidden layers. Then  $\text{NLL}(\boldsymbol{\theta})$  can be formulated as:

$$f = f_4 \circ f_3 \circ f_2 \circ f_1 \quad \left\{ \begin{array}{l} f_1 : x \rightarrow z_1 \\ f_2 : z_1 \rightarrow z_2 \\ f_3 : z_2 \rightarrow \mu \\ f_4 : \mu \rightarrow \text{NLL}(\boldsymbol{\theta}) \end{array} \right.$$

# Backpropagation

## Backpropagation

Backpropagation is an algorithm to compute the gradient of a loss function applied to the output of the network with respect to the parameters in each layer.

## Forward vs Reverse Mode Differentiation

Consider mapping  $\mathbf{o} = \mathbf{f}(\mathbf{x})$  where  $\mathbf{x} \in \mathbb{R}^n$  and  $\mathbf{o} \in \mathbb{R}^m$  is defined as:

$$\mathbf{f} = \mathbf{f}_4 \circ \mathbf{f}_3 \circ \mathbf{f}_2 \circ \mathbf{f}_1, \quad \left\{ \begin{array}{ll} \mathbf{f}_1 : \mathbb{R}^n \rightarrow \mathbb{R}^{m_1} & \mathbf{x}_2 = \mathbf{f}_1(\mathbf{x}) \\ \mathbf{f}_2 : \mathbb{R}^{m_1} \rightarrow \mathbb{R}^{m_2} & \mathbf{x}_3 = \mathbf{f}_2(\mathbf{x}_2) \\ \mathbf{f}_3 : \mathbb{R}^{m_2} \rightarrow \mathbb{R}^{m_3} & \mathbf{x}_4 = \mathbf{f}_3(\mathbf{x}_3) \\ \mathbf{f}_4 : \mathbb{R}^{m_3} \rightarrow \mathbb{R}^m & \mathbf{o} = \mathbf{f}_4(\mathbf{x}_4) \end{array} \right.$$

Using the chain rule, we have:

$$\begin{aligned} \frac{\partial \mathbf{o}}{\partial \mathbf{x}} &= \frac{\partial \mathbf{o}}{\partial \mathbf{x}_4} \frac{\partial \mathbf{x}_4}{\partial \mathbf{x}_3} \frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2} \frac{\partial \mathbf{x}_2}{\partial \mathbf{x}} \\ &= \mathbf{J}_{\mathbf{f}_4}(\mathbf{x}_4) \mathbf{J}_{\mathbf{f}_3}(\mathbf{x}_3) \mathbf{J}_{\mathbf{f}_2}(\mathbf{x}_2) \mathbf{J}_{\mathbf{f}_1}(\mathbf{x}) = \mathbf{J}_{\mathbf{f}}(\mathbf{x}) \in \mathbb{R}^{m \times n} \end{aligned}$$

## Forward vs Reverse Mode Differentiation

$\mathbf{J}_f(\mathbf{x})$  matrix can be written in term of columns and row vectors as:

$$\mathbf{J}_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} = \begin{bmatrix} - & \nabla f_1(\mathbf{x})^T & - \\ & \vdots & \\ - & \nabla f_m(\mathbf{x})^T & - \end{bmatrix} = \begin{bmatrix} \left| \frac{\partial f}{\partial x_1} \right| & \cdots & \left| \frac{\partial f}{\partial x_n} \right| \end{bmatrix} \in \mathbb{R}^{m \times n}$$

- Reverse Mode Differentiation: Assume  $\mathbf{e}_i \in \mathbb{R}^m$  to be the unit basis vector. Then the  $i$ -th row from  $\mathbf{J}_f(\mathbf{x})$  can be extracted by using vector Jacobian product as:

$$\nabla f_i(\mathbf{x})^T = \mathbf{e}_i^T \mathbf{J}_f(\mathbf{x}) = \mathbf{e}_i^T \mathbf{J}_{f_4}(\mathbf{x}_4) \mathbf{J}_{f_3}(\mathbf{x}_3) \mathbf{J}_{f_2}(\mathbf{x}_2) \mathbf{J}_{f_1}(\mathbf{x})$$

- Forward Mode Differentiation: Assume  $\mathbf{e}_j \in \mathbb{R}^n$  to be the unit basis vector. Then the  $j$ -th row from  $\mathbf{J}_f(\mathbf{x})$  can be extracted by using vector Jacobian product as:

$$\frac{\partial f}{\partial x_j} = \mathbf{J}_f(\mathbf{x}) \mathbf{e}_j = \mathbf{J}_{f_4}(\mathbf{x}_4) \mathbf{J}_{f_3}(\mathbf{x}_3) \mathbf{J}_{f_2}(\mathbf{x}_2) \mathbf{J}_{f_1}(\mathbf{x}) \mathbf{e}_j$$

# Forward Mode Differentiation

## Forward Mode Differentiation (FMD)

In forward mode differentiation, we are interested in computing each column of  $\mathbf{J}_f(\mathbf{x})$  at query point  $\mathbf{x}_q$ .

- When  $n < m$ , then it is efficient to use FMD.

---

### Algorithm 1: Forward Mode Differentiation

---

**Initialization:**  $\mathbf{x}_1 = \mathbf{x}_q$

$$\mathbf{v}_j = \mathbf{e}_j \in \mathbb{R}^n, \quad j = 1, \dots, n$$

**begin**

**for**  $k = 1 : K$  **do**

$\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k)$

**for**  $j = 1 : n$  **do**

$\mathbf{v}_j = \mathbf{J}_{\mathbf{f}_k}(\mathbf{x}_k)\mathbf{v}_j$

**end**

**end**

**end**

**Output**           :  $\mathbf{o} = \mathbf{x}_{K+1}, \mathbf{J}_f(\mathbf{x}_q) = [\mathbf{v}_1, \dots, \mathbf{v}_n]$

---

## Forward Mode Differentiation

Consider the following functions:

$$\mathbf{f}_1 : \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \rightarrow \begin{bmatrix} x_1 x_2 \\ x_1 + x_2 \end{bmatrix}, \mathbf{f}_2 : \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \rightarrow \begin{bmatrix} x_1 x_2^2 \\ x_1^2 + x_2^2 \\ \frac{x_1}{x_2} \end{bmatrix}$$

Assume  $\mathbf{f}(\mathbf{x}) = \mathbf{f}_2 \circ \mathbf{f}_1$ . Compute  $\mathbf{J}_{\mathbf{f}}(\mathbf{x}_q)$  for  $\mathbf{x}_q = [1, 1]^T$ .

*Solution:* In this example,  $m = 3$  and  $n = 2$ . Thus  $\mathbf{J}_{\mathbf{f}}(\mathbf{x}_q) \in \mathbb{R}^{3 \times 2}$  and we have the following initializations:

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{v}_1 = \mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \mathbf{v}_2 = \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

We also have:

$$\mathbf{J}_{\mathbf{f}_1} \left( \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right) = \begin{bmatrix} x_2 & x_1 \\ 1 & 1 \end{bmatrix}, \mathbf{J}_{\mathbf{f}_2} \left( \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right) = \begin{bmatrix} x_2^2 & 2x_1 x_2 \\ 2x_1 & 2x_2 \\ \frac{1}{x_2} & -\frac{x_1}{x_2^2} \end{bmatrix}$$

## Forward Mode Differentiation

- $k = 1$ :

$$\mathbf{x}_2 = \mathbf{f}_1(\mathbf{x}_1) = \mathbf{f}_1\left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{J}_{\mathbf{f}_1}\left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\mathbf{v}_1^{new} = \mathbf{J}_{\mathbf{f}_1}\left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) \mathbf{v}_1^{old} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\mathbf{v}_2^{new} = \mathbf{J}_{\mathbf{f}_1}\left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) \mathbf{v}_2^{old} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

## Forward Mode Differentiation

- $k = 2$ :

$$\mathbf{x}_3 = \mathbf{f}_2(\mathbf{x}_2) = \mathbf{f}_2\left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right) = \begin{bmatrix} 4 \\ 5 \\ 0.5 \end{bmatrix}, \quad \mathbf{J}_{\mathbf{f}_2}\left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right) = \begin{bmatrix} 4 & 4 \\ 2 & 4 \\ 0.5 & -0.25 \end{bmatrix}$$

$$\mathbf{v}_1^{new} = \mathbf{J}_{\mathbf{f}_2}\left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right) \mathbf{v}_1^{old} = \begin{bmatrix} 4 & 4 \\ 2 & 4 \\ 0.5 & -0.25 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 8 \\ 6 \\ 0.25 \end{bmatrix}$$

$$\mathbf{v}_2^{new} = \mathbf{J}_{\mathbf{f}_2}\left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right) \mathbf{v}_2^{old} = \begin{bmatrix} 4 & 4 \\ 2 & 4 \\ 0.5 & -0.25 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 8 \\ 6 \\ 0.25 \end{bmatrix}$$

Thus we have:

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x}_q) = [\mathbf{v}_1, \dots, \mathbf{v}_n] = \begin{bmatrix} 8 & 8 \\ 6 & 6 \\ 0.25 & 0.25 \end{bmatrix}$$

# Reverse Mode Differentiation

## Reverse Mode Differentiation (RMD)

In reverse mode differentiation, we are interested in computing each row of  $\mathbf{J}_f(\mathbf{x})$  at query point  $\mathbf{x}_q$ .

- When  $m < n$ , then it is efficient to use RMD.

---

### Algorithm 2: Reverse Mode Differentiation

---

**Initialization:**  $\mathbf{x}_1 = \mathbf{x}_q$   
 $\mathbf{u}_i = \mathbf{e}_i \in \mathbb{R}^m, j = 1, \dots, m$

```
begin
  for  $k = 1 : K$  do
    |  $\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k)$ 
  end
  for  $k = K : 1$  do
    | for  $i = 1 : m$  do
    | |  $\mathbf{u}_i^{T,new} = \mathbf{u}_i^{T,old} \mathbf{J}_{f_k}(\mathbf{x}_k)$ 
    | end
  end
end
end
```

**Output** :  $\mathbf{o} = \mathbf{x}_{K+1}, \mathbf{J}_f(\mathbf{x}_q) = \begin{bmatrix} \mathbf{u}_1^T \\ \vdots \\ \mathbf{u}_m^T \end{bmatrix}$

# Reverse Mode Differentiation

## Reverse Mode Differentiation

Consider our previous functions as:

$$\mathbf{f}_1 : \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \rightarrow \begin{bmatrix} x_1 x_2 \\ x_1 + x_2 \end{bmatrix}, \mathbf{f}_2 : \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \rightarrow \begin{bmatrix} x_1 x_2^2 \\ x_1^2 + x_2^2 \end{bmatrix}$$

Again  $\mathbf{f}(\mathbf{x}) = \mathbf{f}_2 \circ \mathbf{f}_1$ . Compute  $\mathbf{J}_{\mathbf{f}}(\mathbf{x}_q)$  for  $\mathbf{x}_q = [1, 1]^T$ .

*Solution:* In this example,  $m = 3$  and  $n = 2$ . Thus  $\mathbf{J}_{\mathbf{f}}(\mathbf{x}_q) \in \mathbb{R}^{3 \times 2}$  and we have the following initializations:

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{u}_1 = \mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \mathbf{u}_2 = \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{u}_3 = \mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

We also have:

$$\mathbf{J}_{\mathbf{f}_1} \left( \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right) = \begin{bmatrix} x_2 & x_1 \\ 1 & 1 \end{bmatrix}, \mathbf{J}_{\mathbf{f}_2} \left( \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right) = \begin{bmatrix} x_2^2 & 2x_1 x_2 \\ 2x_1 & 2x_2 \\ \frac{1}{x_2} & -\frac{x_1}{x_2^2} \end{bmatrix}$$

## Reverse Mode Differentiation

- Forward loop:

$$\mathbf{x}_2 = \mathbf{f}_1(\mathbf{x}_1) = \mathbf{f}_1\left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \mathbf{x}_3 = \mathbf{f}_2(\mathbf{x}_2) = \mathbf{f}_2\left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right) = \begin{bmatrix} 4 \\ 5 \\ 0.5 \end{bmatrix}$$

- $k = 2$ :

$$\mathbf{J}_{\mathbf{f}_2}(\mathbf{x}_2) = \mathbf{J}_{\mathbf{f}_2}\left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right) = \begin{bmatrix} \frac{4}{2} & \frac{4}{4} \\ 0.5 & -0.25 \end{bmatrix}$$

$$\mathbf{u}_1^{T,new} = \mathbf{u}_1^{T,old} \mathbf{J}_{\mathbf{f}_2}\left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right) = [1 \quad 0 \quad 0] \begin{bmatrix} \frac{4}{2} & \frac{4}{4} \\ 0.5 & -0.25 \end{bmatrix} = [4 \quad 4]$$

$$\mathbf{u}_2^{T,new} = \mathbf{u}_2^{T,old} \mathbf{J}_{\mathbf{f}_2}\left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right) = [0 \quad 1 \quad 0] \begin{bmatrix} \frac{4}{2} & \frac{4}{4} \\ 0.5 & -0.25 \end{bmatrix} = [2 \quad 4]$$

$$\mathbf{u}_3^{T,new} = \mathbf{u}_3^{T,old} \mathbf{J}_{\mathbf{f}_2}\left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right) = [0 \quad 0 \quad 1] \begin{bmatrix} \frac{4}{2} & \frac{4}{4} \\ 0.5 & -0.25 \end{bmatrix} = [0.5 \quad -0.25]$$

## Reverse Mode Differentiation

- $k = 1$ :

$$\mathbf{J}_{f_1}(\mathbf{x}_1) = \mathbf{J}_{f_1} \left( \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\mathbf{u}_1^{T,new} = \mathbf{u}_1^{T,old} \mathbf{J}_{f_1} \left( \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 4 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 8 & 8 \end{bmatrix}$$

$$\mathbf{u}_2^{T,new} = \mathbf{u}_2^{T,old} \mathbf{J}_{f_1} \left( \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 6 \end{bmatrix}$$

$$\mathbf{u}_3^{T,new} = \mathbf{u}_3^{T,old} \mathbf{J}_{f_1} \left( \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 0.5 & -0.25 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 0.25 & 0.25 \end{bmatrix}$$

Thus we have:

$$\mathbf{J}_f(\mathbf{x}_q) = \begin{bmatrix} \mathbf{u}_1^T \\ \vdots \\ \mathbf{u}_m^T \end{bmatrix} = \begin{bmatrix} 8 & 8 \\ 6 & 6 \\ 0.25 & 0.25 \end{bmatrix}$$

## RMD for MLP

To estimate parameters  $\theta$  in MLPs, we have the following optimization problem (for both classification and regression):

$$\hat{\theta}_{mle} = \underset{\theta}{\operatorname{argmin}} \operatorname{NLL}(\theta)$$

where  $\operatorname{NLL}(\theta)$  is a hierarchical mapping. Thus  $m = 1$  and  $n > 1$  and RMD is more efficient than FMD.

## Hierarchical Structure of MLPs

Assume an MLP with one hidden layer for multi-class classification. Then we can write  $\text{NLL}(\boldsymbol{\theta})$  as:

$$\mathcal{L} = \mathbf{f}_4 \circ \mathbf{f}_3 \circ \mathbf{f}_2 \circ \mathbf{f}_1$$

where:

$$\begin{aligned} \mathbf{x}_2 &= \mathbf{f}_1(\mathbf{x}, \mathbf{W}_1, \mathbf{b}_1) = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1 & \mathbf{x}_3 &= \mathbf{f}_2(\mathbf{x}_2) = \varphi(\mathbf{x}_2) \\ \mathbf{x}_4 &= \mathbf{f}_3(\mathbf{x}_3, \boldsymbol{\theta}_3) = \mathbf{W}_2 \mathbf{x}_3 & \mathcal{L} &= \mathbf{f}_4(\mathbf{x}_4, \mathbf{y}) = \mathbb{H}(\mathbf{x}_4, \mathbf{y}) \end{aligned}$$

Thus we can compute the gradient with respect MLP parameters using RMD as:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} &= \frac{\partial \mathcal{L}}{\partial \mathbf{x}_4} \frac{\partial \mathbf{x}_4}{\partial \mathbf{W}_2} & \frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} &= \frac{\partial \mathcal{L}}{\partial \mathbf{x}_2} \frac{\partial \mathbf{x}_2}{\partial \mathbf{W}_1} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} &= \frac{\partial \mathcal{L}}{\partial \mathbf{x}_2} \frac{\partial \mathbf{x}_2}{\partial \mathbf{b}_1} \end{aligned}$$

# Backpropagation Algorithm

---

**Algorithm 3:** Backpropagation for an MLP with  $K$  layers

---

**Initialization:**  $x_1 = x$

**begin**

**for**  $k = 1 : K$  **do**

$x_{k+1} = f_k(x_k, \theta_k)$

**end**

$u_{K+1} = 1$

**for**  $k = K : 1$  **do**

**for**  $i = 1 : m$  **do**

$g_k = u_{k+1}^T \frac{\partial f_k(x_k, \theta_k)}{\partial \theta_k}$

$u_k^T = u_{k+1}^T \frac{\partial f_k(x_k, \theta_k)}{\partial x_k}$

**end**

**end**

**end**

**Output**           :  $\mathcal{L} = x_{K+1}$   
                     $\nabla_x \mathcal{L} = u_1$   
                     $\{\nabla_{\theta_k} \mathcal{L} = g_k : k = 1 : K\}$

---

## Cross Entropy Layer

- If we define  $\mathbf{p} = \mathcal{S}(\mathbf{x})$  then the Mapping is:

$$z = f(\mathbf{x}) = \mathbb{H}(\mathbf{y}, \mathbf{x}) = - \sum_c y_c \log(\mathcal{S}(\mathbf{x})_c) = - \sum_c y_c \log p_c$$

where  $m = 1$ ,  $n = C$  and  $\mathbf{J}_f(\mathbf{x}) \in \mathbb{R}^{1 \times C}$ .

- Assume the target label is  $c$ , then:

$$z = f(\mathbf{x}) = -\log(p_c) = -\log\left(\frac{e^{x_c}}{\sum_j e^{x_j}}\right) = \log\left(\sum_j e^{x_j}\right) - x_c$$

$$\frac{\partial z}{\partial x_i} = \frac{\partial}{\partial x_i} \log \sum_j e^{x_j} - \frac{\partial}{\partial x_i} x_c = \frac{e^{x_i}}{\sum_j e^{x_j}} - \mathbb{I}(i = c)$$

$$\Rightarrow \mathbf{J}_f(\mathbf{x}) = (\mathbf{p} - \mathbf{y})^T$$

## Elementwise Nonlinearity

- The Mapping is:

$$\mathbf{z} = \mathbf{f}(\mathbf{x}) = \varphi(\mathbf{x}) \Rightarrow z_i = \varphi(x_i), \quad i = 1, \dots, p$$

where  $m = p$ ,  $n = p$  and  $\mathbf{J}_f(\mathbf{x}) \in \mathbb{R}^{p \times p}$ .

- The  $(i, j)$  element of Jacobian matrix is:

$$\frac{\partial z_i}{\partial x_j} = \begin{cases} \varphi'(x_i) & \text{if } i = j \\ 0 & \text{otherwise} \end{cases} \Rightarrow \mathbf{J}_f(\mathbf{x}) = \text{diag}(\varphi'(\mathbf{x}))$$

## Linear layer

- The Mapping is:

$$\mathbf{z} = \mathbf{f}(\mathbf{x}, \mathbf{W}, \mathbf{b}) = \mathbf{W}\mathbf{x} + \mathbf{b}$$

where  $\mathbf{x} \in \mathbb{R}^n$ ,  $\mathbf{z} \in \mathbb{R}^m$  and  $\mathbf{J}_f(\mathbf{x}) = \frac{\partial \mathbf{z}}{\partial \mathbf{x}} \in \mathbb{R}^{m \times n}$ .

- We know that  $z_i = \sum_{k=1}^n W_{ik}x_k$ , thus  $(i, j)$  element of Jacobian matrix is:

$$\frac{\partial z_i}{\partial x_j} = \frac{\partial}{\partial x_j} \sum_{k=1}^n W_{ik}x_k = \sum_{k=1}^n W_{ik} \frac{\partial}{\partial x_j} x_k = \sum_{k=1}^n W_{ik} \mathbb{I}(k = j) = W_{ij}$$

$$\Rightarrow \mathbf{J}_f(\mathbf{x}) = \mathbf{W}$$

## Linear layer (Continue)

- Calculating  $\frac{\partial \mathcal{L}}{\partial \text{vec}(\mathbf{W})} = \mathbf{u}^T \frac{\partial \mathbf{z}}{\partial \text{vec}(\mathbf{W})}$  where  $\mathbf{u} \in \mathbb{R}^m$  and  $\frac{\partial \mathbf{z}}{\partial \text{vec}(\mathbf{W})} \in \mathbb{R}^{m \times (m \times n)}$   
First, we calculate an arbitrary column in  $\frac{\partial \mathbf{z}}{\partial \text{vec}(\mathbf{W})}$  vector:

$$z_k = \sum_{l=1}^m W_{kl} x_l + b_k \Rightarrow \frac{\partial z_k}{\partial W_{ij}} = \sum_{l=1}^m x_l \frac{\partial}{\partial W_{ij}} W_{kl} = \sum_{l=1}^m x_l \mathbb{I}(i = k, j = l)$$
$$\Rightarrow \frac{\partial \mathbf{z}}{\partial W_{ij}} = x_j \times \mathbf{e}_i = (0, \dots, x_j, \dots, 0)^T \in \mathbb{R}^m$$

Thus the corresponding column in  $\frac{\partial \mathcal{L}}{\partial \text{vec}(\mathbf{W})}$  is:

$$\mathbf{u}^T \frac{\partial \mathbf{z}}{\partial W_{ij}} = \sum_{k=1}^m u_k \frac{\partial z_k}{\partial W_{ij}} = u_i x_j$$

If we use inverse vectorizing operator, we have:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{u} \mathbf{x}^T \in \mathbb{R}^{m \times n}$$

## Linear layer (Continue)

- Calculating  $\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \mathbf{u}^T \frac{\partial \mathbf{z}}{\partial \mathbf{b}}$  where  $\mathbf{u} \in \mathbb{R}^m$  and  $\frac{\partial \mathbf{z}}{\partial \mathbf{b}} \in \mathbb{R}^{m \times m}$   
We know:

$$z_k = \sum_{l=1}^m W_{kl} x_l + b_k \Rightarrow \frac{\partial z_k}{\partial b_j} = \frac{\partial}{\partial b_j} b_k = \mathbb{I}(j = k) \Rightarrow \frac{\partial \mathbf{z}}{\partial \mathbf{b}} = \mathbf{I} \in \mathbb{R}^{m \times m}$$

Thus we have:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \mathbf{u}^T \frac{\partial \mathbf{z}}{\partial \mathbf{b}} = \mathbf{u}^T \mathbf{I} = \mathbf{u}^T$$

# References I



Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner,  
“Gradient-based learning applied to document recognition,”  
*Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.